

Práctica 10. Refactorización de código Java.

Juego Buscatesoros

Se ha creado un programa en Java que simula un juego de búsqueda de tesoros en un tablero de casillas. El código ha sido desarrollado por Don Becario nuestro nuevo compañero de la oficina, y tiene muchos errores así que debes refactorizarlo.

El juego consiste en que el jugador intenta encontrar todos los tesoros escondidos en el tablero, teniendo en cuenta que tiene un máximo de X intentos.

Requisitos funcionales

- Representar el tablero como una matriz de casillas
- Casilla es una clase con un atributo boolean tesoro y un atributo boolean visitada.
- Algunos elementos del tablero representan tesoros. Colocar 5 tesoros de forma aleatoria en el tablero en casillas diferentes
- El jugador elige una casilla indicando fila y columna.
- Si la casilla contiene un tesoro:
 - Se muestra un mensaje "Tesoro encontrado!".
 - Se incrementa el contador de tesoros encontrados.
 - La casilla se marca como visitada
- Si no hay tesoro:
 - Se muestra "Nada aquí...".
 - La casilla se marca como visitada.
- Hay un máximo número de intentos, si se supera se termina el juego y aparece "Has perdido"
- Si se encuentran todos los tesoros antes de superar el máximo número de intentos se termina el juego y aparece "Has ganado"

Un ejemplo del juego puesto en marcha:

Iniciando juego...

Introduce fila:

1

Introduce columna:

1

Nada aquí...

```
-----  
- X ---  
-----  
-----  
-----
```

Introduce fila:

3

Introduce columna:

2

Nada aquí...

```
-----  
- X ---  
-----  
-- X --  
-----
```

Introduce fila:

4

Introduce columna:

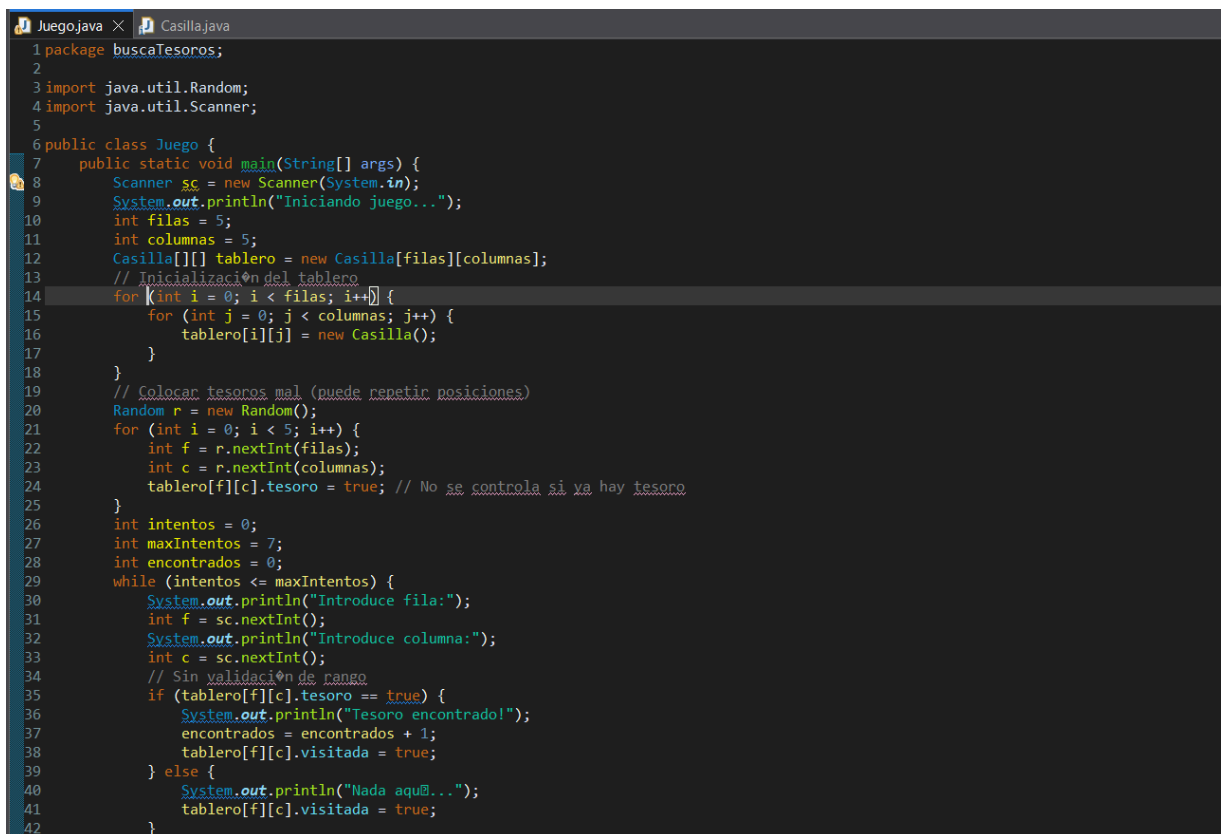
4

Tesoro encontrado!

```
-----  
- X ---  
-----  
-- X --  
----- O
```

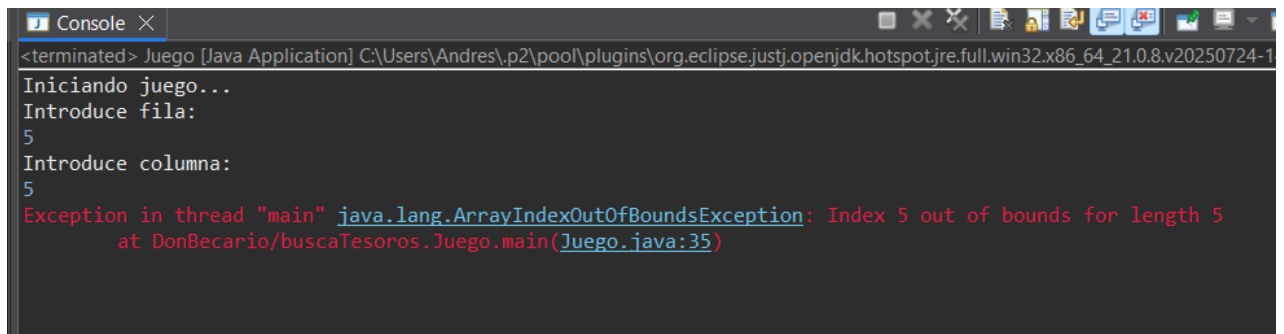
Tareas a realizar:

1) Copia el código de Don Becario a Eclipse.



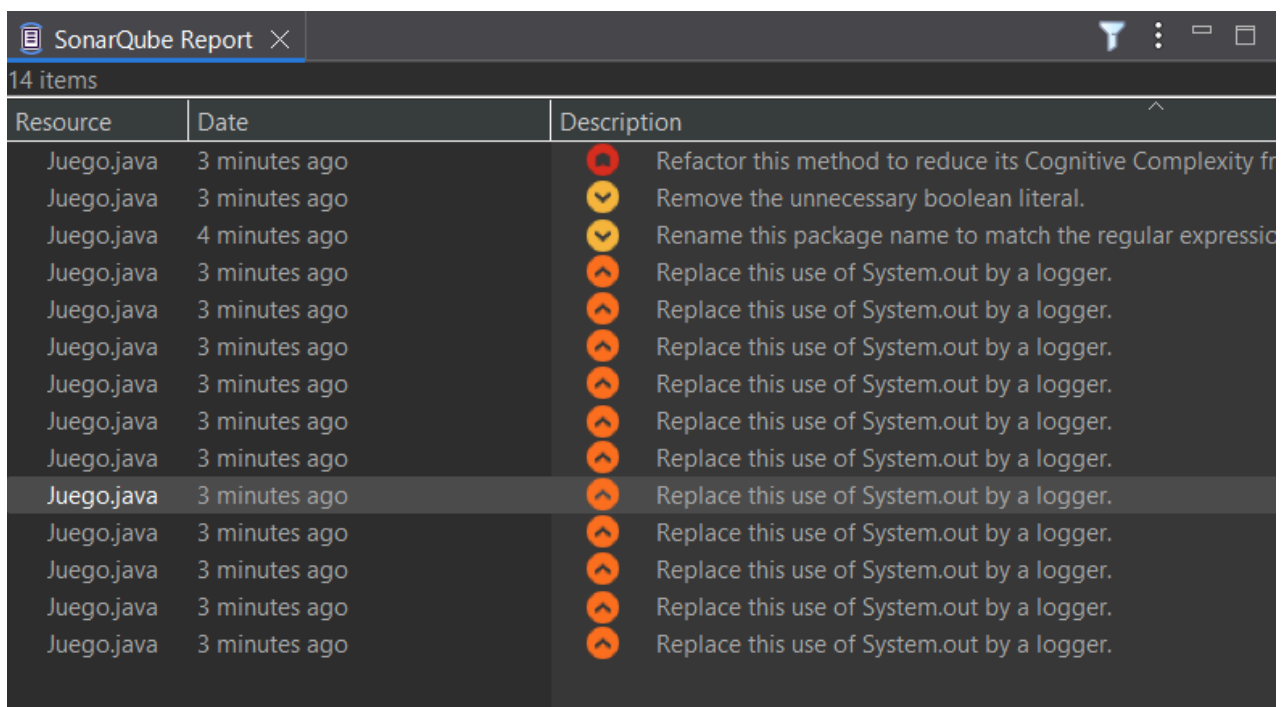
```
1 package buscaTesoros;  
2  
3 import java.util.Random;  
4 import java.util.Scanner;  
5  
6 public class Juego {  
7     public static void main(String[] args) {  
8         Scanner sc = new Scanner(System.in);  
9         System.out.println("Iniciando juego...");  
10        int filas = 5;  
11        int columnas = 5;  
12        Casilla[][] tablero = new Casilla[filas][columnas];  
13        // Inicialización del tablero  
14        for (int i = 0; i < filas; i++) {  
15            for (int j = 0; j < columnas; j++) {  
16                tablero[i][j] = new Casilla();  
17            }  
18        }  
19        // Colocar tesoros mal (puede repetir posiciones)  
20        Random r = new Random();  
21        for (int i = 0; i < 5; i++) {  
22            int f = r.nextInt(filas);  
23            int c = r.nextInt(columnas);  
24            tablero[f][c].tesoro = true; // No se controla si ya hay tesoro  
25        }  
26        int intentos = 0;  
27        int maxIntentos = 7;  
28        int encontrados = 0;  
29        while (intentos <= maxIntentos) {  
30            System.out.println("Introduce fila:");  
31            int f = sc.nextInt();  
32            System.out.println("Introduce columna:");  
33            int c = sc.nextInt();  
34            // Sin validación de rango  
35            if (tablero[f][c].tesoro == true) {  
36                System.out.println("Tesoro encontrado!");  
37                encontrados = encontrados + 1;  
38                tablero[f][c].visitada = true;  
39            } else {  
40                System.out.println("Nada aquí...");  
41                tablero[f][c].visitada = true;  
42            }  
43        }  
44    }  
45 }
```

2) Compila y ejecuta el programa



```
<terminated> Juego [Java Application] C:\Users\Andres\AppData\Local\Temp\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-1
Iniciando juego...
Introduce fila:
5
Introduce columna:
5
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
    at DonBecario/buscaTesoros.Juego.main(Juego.java:35)
```

3) Ejecuta SonarQube para que analice ambos ficheros. El fichero Juego.java y Casilla.java que contiene la práctica presenta varias advertencias de SonarQube, entre ellas:



Resource	Date	Description
Juego.java	3 minutes ago	Refactor this method to reduce its Cognitive Complexity from 27 to the 15 allowed.
Juego.java	3 minutes ago	Remove the unnecessary boolean literal.
Juego.java	4 minutes ago	Rename this package name to match the regular expression
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.
Juego.java	3 minutes ago	Replace this use of System.out by a logger.

“Refactor this method to reduce its Cognitive Complexity from 27 to the 15 allowed.”

Esto significa que tu método alcanzó una complejidad cognitiva de 27, pero la regla del proyecto establece que el límite permitido es 15.

Un método con complejidad cognitiva alta:

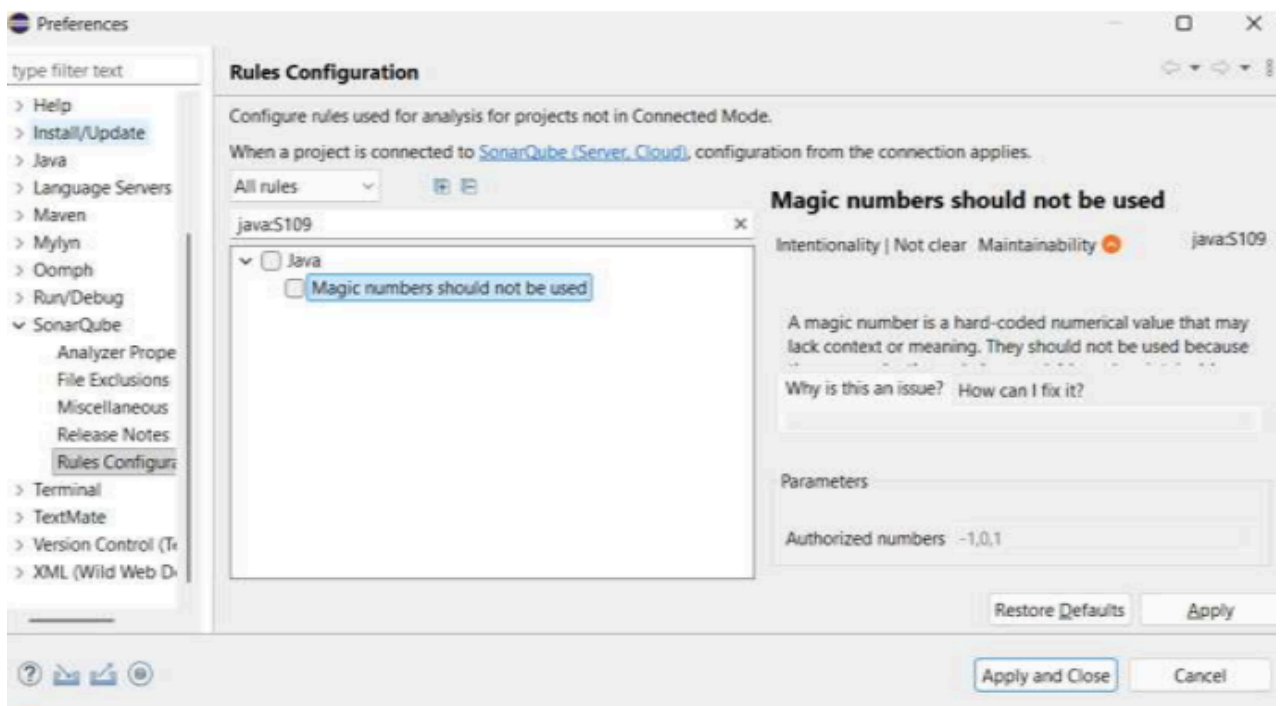
- Es difícil de leer
- Es difícil de mantener
- Es más propenso a errores
- Cuesta más testarlo

Por lo tanto, SonarQube te dice que refactorices el método para:

- Dividirlo en métodos más pequeños
- Simplificar la lógica
- Reducir anidamiento
- Mejorar legibilidad

“Assign this magic number 5 to a well-named constant, and use the constant instead.”

Ojo, si Sonar no te indica ese error debes activarlo en las reglas (java:S109)



4) Refactoriza el código para el código para eliminar ambas notificaciones, manteniendo intacta la, manteniendo intacta la funcionalidad del juego. Para ello debes:

Para ello debes:

1. Reducir la complejidad cognitiva del método

Reducir la complejidad cognitiva del método indicado por SonarQube hasta un valor indicado por SonarQube hasta un valor igual o inferior a 15.

- Debes extraer nuevos
nuevos métodos más pequeños, reorganizar lógica, reducir

, reorganizar lógica, reducir
anidamientos, eliminar duplicación, etc.
anidamientos, eliminar duplicación, etc.

- limpiarTablero()
limpiarTablero()

```
private void limpiarTablero() {  
    for (int i = 0; i < TAMAÑO; i++) {  
        for (int j = 0; j < TAMAÑO; j++) {  
            tablero[i][j] = new Casilla();  
        }  
    }  
}
```

- ponerTesoros()
ponerTesoros()

```
private void ponerTesoros() {  
    Random r = new Random();  
    int colocados = 0;  
    while (colocados < NUM_TESOROS) {  
        int f = r.nextInt(TAMAÑO);  
        int c = r.nextInt(TAMAÑO);  
        if (!tablero[f][c].isTesoro()) {  
            tablero[f][c].setTesoro(true);  
            colocados++;  
        }  
    }  
}
```

- verificarCasilla()
verificarCasilla()

```
private void verificarCasilla(int f, int c) {  
    if (tablero[f][c].isTesoro()) {  
        System.out.println("Tesoro encontrado!");  
        tesorosEncontrados++;  
    } else {  
        System.out.println("Nada aqui...");  
    }  
    tablero[f][c].setVisitada(true);  
}
```

- Encapsular atributos de la clase (Encapsular atributos de la clase (tablero, intentos, tesorosEncontrados tesorosEncontrados) y usar getters y setters.
getters y setters.

```
public Casilla[][] getTablero() {
    return tablero;
}
public int getIntentos() {
    return intentos;
}
public int getTesorosEncontrados() {
    return tesorosEncontrados;
}
```

2. Eliminar el uso de números mágicos (como el valor 5)
Eliminar el uso de números mágicos (como el valor 5) convirtiéndolos en
convirtiéndolos en constantes
con nombres expresivos
con nombres expresivos, puedes ponerlos como atributos de la clase
puedes ponerlos como atributos de la clase por ejemplo:

- Tamaño del tablero.
Tamaño del tablero.
`private static final int TAMAÑO = 5;`
- Número de tesoros.
Número de tesoros.
`private static final int NUM_TESOROS = 5;`
- Valor de la casilla de tesoro.
Valor de la casilla de tesoro.
`private static final String SIMBOLO_TESORO = " O ";`
- Valor de la casilla visitada.
Valor de la casilla visitada.
`private static final String SIMBOLO_VISITADA = " X ";`

3. Mantener la funcionalidad completa del juego
Mantener la funcionalidad completa del juego, sin modificar el comportamiento
sin modificar el comportamiento
esperado.

```
- - - - -
X - - - -
Introduce fila:
1
Introduce columna:
0
Nada aqui...
X X X X X
X - - - -
- - - - -
- - - - -
X - - - -
Has perdido
```

4. Aplicar buenas prácticas de programación

Aplicar buenas prácticas de programación:

- Código limpio y legible.
Código limpio y legible.
- Métodos cortos y cohesivos.
Métodos cortos y cohesivos.
- Nombres descriptivos.
Nombres descriptivos.

Código nuevo:

```
package buscaTesoros;
import java.util.Random;
import java.util.Scanner;
public class Juego {
    private static final int TAMAÑO = 5;
    private static final int NUM_TESOROS = 5;
    private static final int MAX_INTENTOS = 7;
    private static final String SIMBOLO_TESORO = " O ";
    private static final String SIMBOLO_VISITADA = " X ";
    private static final String SIMBOLO_VACIA = " - ";
    private Casilla[][] tablero;
    private int intentos;
    private int tesorosEncontrados;
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Iniciando juego...");
        Juego juego = new Juego();
        juego.tablero = new Casilla[TAMAÑO][TAMAÑO];
        juego.intentos = 0;
        juego.tesorosEncontrados = 0;
        juego.limpiarTablero();
        juego.ponerTesoros();
        while (juego.intentos < MAX_INTENTOS && juego.tesorosEncontrados <
NUM_TESOROS) {
            System.out.println("Introduce fila:");
            int f = sc.nextInt();
            System.out.println("Introduce columna:");
            int c = sc.nextInt();
            if (f < 0 || f >= TAMAÑO || c < 0 || c >= TAMAÑO) {
                System.out.println("Esa casilla no existe, prueba otra vez.");
                continue;
            }
            juego.verificarCasilla(f, c);
            juego.intentos++;
            juego.imprimirTablero();
        }
        if (juego.tesorosEncontrados == NUM_TESOROS) {
            System.out.println("Has ganado");
        } else {
            System.out.println("Has perdido");
        }
        sc.close();
    }
    private void limpiarTablero() {
        for (int i = 0; i < TAMAÑO; i++) {
            for (int j = 0; j < TAMAÑO; j++) {
```

```

        tablero[i][j] = new Casilla();
    }
}

private void ponerTesoros() {
    Random r = new Random();
    int colocados = 0;
    while (colocados < NUM_TESOROS) {
        int f = r.nextInt(TAMAÑO);
        int c = r.nextInt(TAMAÑO);
        if (!tablero[f][c].isTesoro()) {
            tablero[f][c].setTesoro(true);
            colocados++;
        }
    }
}

private void verificarCasilla(int f, int c) {
    if (tablero[f][c].isTesoro()) {
        System.out.println("Tesoro encontrado!");
        tesorosEncontrados++;
    } else {
        System.out.println("Nada aqui...");
    }
    tablero[f][c].setVisitada(true);
}

private void imprimirTablero() {
    for (int i = 0; i < TAMAÑO; i++) {
        for (int j = 0; j < TAMAÑO; j++) {
            if (!tablero[i][j].isVisitada()) {
                System.out.print(SIMBOLO_VACIA);
            } else if (tablero[i][j].isTesoro()) {
                System.out.print(SIMBOLO_TESORO);
            } else {
                System.out.print(SIMBOLO_VISITADA);
            }
        }
        System.out.println();
    }
}

public Casilla[][] getTablero() {
    return tablero;
}

public int getIntentos() {
    return intentos;
}

public int getTesorosEncontrados() {
    return tesorosEncontrados;
}
}

```



```

package buscaTesoros;
public class Casilla {
    private boolean tesoro;
    private boolean visitada;
    public boolean isTesoro() {
        return tesoro;
    }
    public void setTesoro(boolean tesoro) {
        this.tesoro = tesoro;
    }
    public boolean isVisitada() {
        return visitada;
    }
    public void setVisitada(boolean visitada) {
        this.visitada = visitada;
    }
}

```

Entrega

- Súbe tu código a Github refactorizado y probado y envía la dirección del repositorio al profesor.

<https://github.com/AndresCordobaEnlaces/Practica10Refactorizacion>

Código de Don Becario

```

package buscaTesoros;
import java.util.Random;
import java.util.Scanner;
public class Juego {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Iniciando juego...");
        int filas = 5;
        int columnas = 5;
        Casilla[][] tablero = new Casilla[filas][columnas];
        // Inicialización del tablero
        for (int i = 0; i < filas; i++) {
            for (int j = 0; j < columnas; j++) {
                tablero[i][j] = new Casilla();
            }
        }
        // Colocar tesoros mal (puede repetir posiciones)
        Random r = new Random();
        for (int i = 0; i < 5; i++) {
            int f = r.nextInt(filas);
            int c = r.nextInt(columnas);
            tablero[f][c].tesoro = true; // No se controla si ya hay tesoro
        }
        int intentos = 0;
        int maxIntentos = 7;
        int encontrados = 0;
        while (intentos <= maxIntentos) {
            System.out.println("Introduce fila:");
            int f = sc.nextInt();
            System.out.println("Introduce columna:");
            int c = sc.nextInt();
            // Sin validación de rango

```

```

        if (tablero[f][c].tesoro == true) {
            System.out.println("Tesoro encontrado!");
            encontrados = encontrados + 1;
            tablero[f][c].visitada = true;
        } else {
            System.out.println("Nada aqu\u2666...");
            tablero[f][c].visitada = true;
        }
        intentos++;
        // Imprime el tablero
        for (int i = 0; i < tablero.length; i++) {
            for (int j = 0; j < tablero[i].length; j++) {
                if (tablero[i][j].visitada) {
                    if (tablero[i][j].tesoro) {
                        System.out.print(" O ");
                    } else {
                        System.out.print(" X ");
                    }
                } else {
                    System.out.print(" - ");
                }
            }
            System.out.println();
        }
        if (encontrados == 5) {
            System.out.println("Has ganado");
            break;
        }
    }
    if (intentos > maxIntentos) {
        System.out.println("Has perdido");
    }
}

```

```

package buscaTesoros;
public class Casilla {
    public boolean tesoro;
    public boolean visitada;
}

```